A D PATEL INSTITUTE OF TECHNOLOGY

MINI PROJECT REPORT

CAMPUSINDEX: STUDENT RECORD MANAGEMENT & ALGORITHM LAB

Subject: Design and Analysis of Algorithms (DAA)
Semester: V

Submitted by: Jenil Patel

Enrollment Number: 12402080601053

Institute: A D Patel Institute of Technology (ADIT)

Academic Year: 2026–27

TABLE OF CONTENTS

1. ABSTRACT
2. 1. INTRODUCTION
3. 2. PROBLEM STATEMENT
4. 3. OBJECTIVES
5. 4. SCOPE OF THE PROJECT
6. 5. SYSTEM REQUIREMENTS
7. 6. SYSTEM MODULES
8. 7. ALGORITHMS USED
9. 8. METHODOLOGY / WORKING
10. 9. COMPLEXITY ANALYSIS
11. 10. TESTING AND EXPECTED RESULTS
12. 11. ADVANTAGES
13. 12. LIMITATIONS
14. 13. FUTURE ENHANCEMENTS
15. 14. CONCLUSION
16. REFERENCES

ABSTRACT

This mini project, CampusIndex: Student Record Management & Algorithm Lab, is a browser-based educational application designed to manage student records and demonstrate the practical behaviour of searching and sorting algorithms. The system allows a user to add and manage student details such as name, student ID, department, year, CGPA, and marks. It also provides a search interface, sorting visualizer, search comparison module, benchmark area, complexity reference, and CSV export option.

The central academic purpose is to connect Design and Analysis of Algorithms concepts with a practical interface. Linear Search and Binary Search are compared to show how the number of comparisons changes as the dataset grows. The sorting studio allows users to choose a sorting algorithm and a field to sort by, then observe comparisons, writes/swaps, elapsed time, and complexity. The application is intended for learning and demonstration; the deployed page indicates that records are saved locally in the browser workspace.

1. INTRODUCTION

Student information often needs to be searched, sorted, and reviewed according to identifiers or academic performance. When records are handled manually, locating a particular student and arranging records in a required order can take time and can lead to mistakes. A computer-based record interface makes these operations easier to perform and provides a convenient way to demonstrate the algorithms used behind them.

CampusIndex combines a student-record interface with an algorithm learning area. Instead of showing only the final answer, the application is designed to help users explore how different algorithms process the same data and why algorithm choice matters.

2. PROBLEM STATEMENT

To develop an interactive web-based student record system that supports basic record management and demonstrates the working and performance characteristics of searching and sorting algorithms on student data.

3. OBJECTIVES

• Provide a simple interface to add, view, edit, search, and sort student records.

• Represent student information using fields such as student ID, department, year, CGPA, and marks.

• Demonstrate Linear Search and Binary Search and compare their comparison counts.

• Visualize sorting operations and show performance indicators such as comparisons, writes/swaps, and elapsed time.

• Provide benchmark comparisons by running sorting algorithms on the same dataset.

• Explain algorithmic time and space complexity using a reference guide.

• Allow student data to be exported as a CSV file for further use.

4. SCOPE OF THE PROJECT

The project is suitable for academic demonstration, algorithm practice, and small sample datasets. It focuses on record organization and the educational comparison of algorithms rather than a production-grade college database. The visible interface includes a dashboard, student data management, sorting studio, search studio, benchmark lab, and complexity guide.

Based on the deployed interface, the workspace reports that data is saved locally. Therefore, the report does not assume a server database, user authentication, or multi-user synchronization.

5. SYSTEM REQUIREMENTS

Hardware: A desktop, laptop, tablet, or smartphone capable of running a modern web browser.

Software: A modern browser such as Chrome, Edge, Firefox, or Safari; an internet connection to open the deployed GitHub Pages site.

Deployment: GitHub Pages-hosted web application.

Implementation note: The visible application behaves as a client-side web interface. The exact framework, file structure, and implementation details should be confirmed from the source code before being stated as definitive.

6. SYSTEM MODULES

6.1 Dashboard: Presents summary indicators such as total students, average CGPA, top performer, and the stated search complexity. These values depend on the records currently present in the workspace.

6.2 Student Record Management: Lets users add student information and provides controls to search, sort, edit, and manage records. The listed fields include full name, student ID, department, year, CGPA, and marks.

6.3 Sorting Studio: Lets the user select a sorting algorithm, a field, the number of records, and visualization speed. The interface is designed to show records moving through the sort and display comparison counts, writes/swaps, elapsed time, and complexity.

6.4 Search Studio: Compares Linear Search and Binary Search for a selected student. Linear Search checks records one by one; Binary Search repeatedly halves the search range and requires the IDs to be sorted according to the search key.

6.5 Benchmark Lab: Runs sorting algorithms on the same dataset and presents comparative measurements such as algorithm, complexity, comparisons, writes, time, and relative work. Measured time can vary with the browser and device.

6.6 Complexity Guide: Summarizes best, average, and worst-case time complexity, space requirements, and stability for supported algorithms.

6.7 CSV Export: Provides an option to export the student dataset as a CSV file for spreadsheet applications or further analysis.

7. ALGORITHMS USED

7.1 Linear Search: Starts at the first record and checks each record until the target is found or the dataset ends. Its best-case time is O(1), while average- and worst-case time are O(n). It can operate on unsorted data.

7.2 Binary Search: Compares the target with the middle element of a sorted sequence and discards half of the remaining search interval at each step. Its best-case time is O(1), and average- and worst-case time are O(log n). The data must be sorted by the same key used for searching.

7.3 Sorting Algorithms: The sorting studio and benchmark interface advertises six sorting algorithms. The exact names should be checked in the running interface/source code and recorded in the final submission if the faculty requires them. The complexity guide highlights quadratic-time algorithms and more scalable methods such as merge sort, heap sort, and average-case quicksort.

7.4 Big-O Analysis: Big-O notation describes how the amount of work grows with input size. For example, doubling the input size can make an $O(n^2)$ algorithm perform roughly four times as much work, whereas O(n log n) methods generally scale better for large datasets.

8. METHODOLOGY / WORKING

Step 1: Open the deployed CampusIndex application in a modern web browser.

Step 2: Add student records by entering the required identity and academic fields, then save each record.

Step 3: Use the data-management controls to search for a student by name, ID, or department and to arrange records using the available sort controls.

Step 4: Open the Sorting Studio, select an algorithm and sort key, choose the dataset size and speed, and start the visualization.

Step 5: Observe the sequence of comparisons and writes/swaps, elapsed time, and the displayed complexity.

Step 6: In Search Studio, select a target student and run both Linear Search and Binary Search to compare their behaviour and number of comparisons.

Step 7: Run the Benchmark Lab to compare sorting algorithms using the same input data. Interpret timings carefully because browser, device, and dataset size affect execution time.

Step 8: Export records to CSV if a copy of the dataset is needed.

9. COMPLEXITY ANALYSIS

Let n represent the number of student records. Linear Search has O(n) average and worst-case time complexity. Binary Search has O(log n) average and worst-case time complexity when the data is sorted by the search key. Common simple sorting algorithms, such as Bubble Sort and Selection Sort, typically require $O(n^2)$ comparisons in average and worst cases. Efficient comparison sorts such as Merge Sort and Heap Sort typically run in O(n log n) time.

These are theoretical growth rates. The benchmark module supplies empirical measurements for the current browser and dataset; such timings should be reported only after actually running the benchmark. No fabricated benchmark numbers are included in this report.

10. TESTING AND EXPECTED RESULTS

Test 1 — Add record: Enter valid student details and save. Expected result: the record appears in the dataset and dashboard values update where applicable.

Test 2 — Search by student ID/name: Enter a value that exists. Expected result: matching record(s) are shown.

Test 3 — Search for a missing record: Search for an ID or name not present. Expected result: the interface indicates that no matching record was found.

Test 4 — Sort by marks or CGPA: Choose a sort key and sorting option. Expected result: records are arranged according to the selected order.

Test 5 — Compare search algorithms: Select a target and run both search methods. Expected result: the interface displays the search results and comparison information.

Test 6 — Run benchmark: Execute the benchmark using the same dataset for the available algorithms. Expected result: the application displays comparative work/time metrics.

Test 7 — Export CSV: Use the export control. Expected result: a CSV file containing the dataset is downloaded.

Note: These are test cases and expected behaviours based on the visible interface. Actual pass/fail status should be recorded after manually testing the deployed project.

11. ADVANTAGES

• Combines student record management with hands-on algorithm learning.

• Makes searching and sorting concepts easier to understand through visual feedback.

• Compares theoretical complexity with measured execution data.

• Provides a CSV export feature for portability.

• Runs through a web browser without requiring a separate desktop installation.

12. LIMITATIONS

• Local browser storage is not equivalent to a centralized database and may not synchronize between devices or browsers.

• Binary Search is only correct when the searched collection is sorted by the relevant key.

• Benchmark timing varies with device performance, browser workload, and dataset size.

• The current scope is educational; authentication, access control, backups, and multi-user data management are not established by the visible page.

• A larger real-world deployment would need validation, persistent server-side storage, and security controls.

13. FUTURE ENHANCEMENTS

• Add secure login and role-based access for students, faculty, and administrators.

• Integrate a server-side database with backup and recovery.

• Add import-from-CSV and stronger validation for IDs, CGPA, and marks.

• Provide a downloadable benchmark report with charts and repeatable test settings.

• Add automated unit tests for sorting, searching, editing, and export features.

• Improve accessibility, mobile responsiveness, and help text for each algorithm.

14. CONCLUSION

CampusIndex demonstrates how fundamental Design and Analysis of Algorithms concepts can be applied to a student record workflow. Its record-management interface gives the algorithms a practical context, while the Sorting Studio, Search Studio, Benchmark Lab, and Complexity Guide help users compare algorithm behaviour. The project reinforces the importance of selecting an algorithm based on input requirements and expected data size. It is useful as an educational mini project and can be extended with stronger persistence, testing, and security features.

REFERENCES

1. CampusIndex — Student Records & Algorithm Lab, deployed project: https://jenilpatel0112.github.io/DAA-Project-Algorithm-Binary-Search-Student-Record-Sorting/

2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. — Introduction to Algorithms, MIT Press.

3. Project interface and visible module descriptions, accessed 23 October 2026.